MODULE

14

Project structure & packaging

From one script to a real project: imports, virtual environments, a CLI, and pip install.

YOU WILL LEARN

Modules, packages, import	`01_modules_and_imports.py`
Command-line tools	`02_cli_with_argparse.py`
pyproject, PyPI, uv, pre-commit	`03_publishing_and_tooling.md`
The tasky package	`tasky_project/`
Interview Q&A	`INTERVIEW.md`

WHY PROJECT STRUCTURE

One file doesn't scale

THE PROBLEM

Your script grew to **1,500 lines**. Two teammates edit it at once, nobody can test one part alone, and "it works on my machine" because everyone has different library versions.

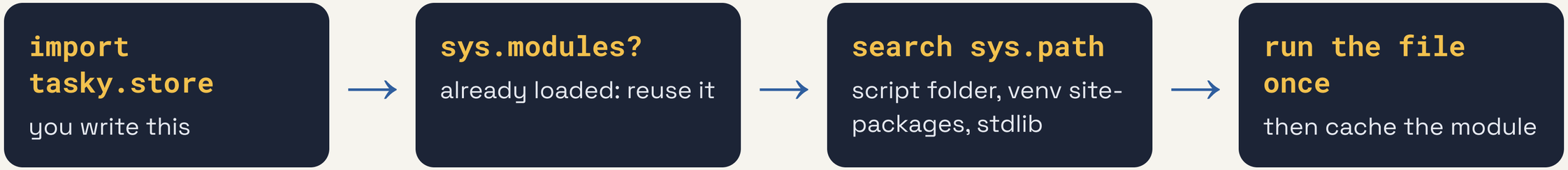
THE IDEA

Split code into modules and packages, pin dependencies in pyproject.toml, and isolate them in a virtual environment.

PICTURE IT IMPORTS

What import really does

📚 Like a **library**: Python looks for the book on a list of shelves (**sys.path**), reads it **once**, and keeps it in a cache (**sys.modules**). The next import just hands you the cached copy.



That's why a module-level object behaves like a **singleton**.

STEP BY STEP PROJECT STRUCTURE

A layout that every team recognises

PATH	WHAT IT'S FOR
`pyproject.toml`	name, version, dependencies, tool settings (ruff, mypy, pytest)
`src/tasky/__init__.py`	makes **tasky** a package; the public API
`src/tasky/__main__.py`	lets you run **python -m tasky**
`src/tasky/cli.py · store.py`	one module per responsibility
`tests/`	pytest tests, mirroring the package
`.venv/` (git-ignored)	this project's own Python + libraries

> The **src/** layout stops tests from accidentally importing local files instead of the installed package.

TRY IT IMPORTS

if __name__ == ”__main__”

```python
# tools.py
def slugify(title):
    return "-".join(title.lower().split())

if __name__ == "__main__":     # only when run directly
    print(slugify("Hello Python Pro"))

print("__name__ is", repr(__name__))
```

OUTPUT

```
hello-python-pro
__name__ is '__main__'
```

1. Run directly: **__name__ == ”__main__”**, so the demo runs.
2. Imported: **__name__ == ”tools”**, so it doesn’t.
3. Put scripts’ entry code behind this guard.

COMMON MISTAKE IMPORTS

Circular imports

✗ WRONG

```
# models.py
from services import send_welcome
class User: ...

# services.py
from models import User      # ImportError:
def send_welcome(u: User): ...  # partially initialised
```

Each module needs the other **while it's still loading**. Python finds a half-built module and fails.

✓ RIGHT

```
# services.py
from __future__ import annotations
from typing import TYPE_CHECKING
if TYPE_CHECKING:            # only for type hints
    from models import User

def send_welcome(u: User): ...
```

Better still: move shared code into a third module, so dependencies point **one way**.

TRY IT COMMAND-LINE TOOLS

argparse: a real CLI in 15 lines

```python
import argparse

parser = argparse.ArgumentParser(prog="tasky", description="Tiny todo app")
sub = parser.add_subparsers(dest="cmd", required=True)

add = sub.add_parser("add", help="add a task")
add.add_argument("title")
add.add_argument("--priority", type=int, choices=[1, 2, 3], default=2)

sub.add_parser("list", help="show tasks")

args = parser.parse_args(["add", "Write slides", "--priority", "1"])
print(args)
print(args.cmd, args.title, args.priority)
```

OUTPUT

```
Namespace(cmd='add', title='Write slides', priority=1)
add Write slides 1
```

1. **Subcommands** like git: **tasky add**, **tasky list**.
2. **type=int** and **choices** validate input for you.
3. **--help** is generated automatically.

STEP BY STEP ENVIRONMENTS AND TOOLS

The modern Python toolbox

NEED	TOOL	COMMAND
Isolate dependencies	venv / uv	`python -m venv .venv` · `uv venv`
Install the project	pip / uv	`pip install -e .` · `uv sync`
Pin exact versions	lock file	`uv lock` · `pip freeze > requirements.txt`
Build and publish	build + twine	`python -m build` · `twine upload dist/*`
Checks on every commit	pre-commit	`pre-commit install` (ruff, mypy)

> One rule: **never pip install into the system Python.** One venv per project.

TRY IT PYPROJECT.TOML

One file describes the project

```
print(open("pyproject.toml").read())
```

OUTPUT

```
[project]
name = "tasky"
version = "1.0.0"
requires-python = ">=3.10"
dependencies = ["rich>=13"]

[project.scripts]
tasky = "tasky.cli:main"      # creates a 'tasky' command

[project.optional-dependencies]
dev = ["pytest", "mypy", "ruff"]
```

1. **[project.scripts]**: pip creates a real **tasky** command.
2. Dev tools: **pip install -e ".[dev]"**.

IN REAL JOBS PACKAGING

On day one at a new job

Set up the project

```
git clone ... && cd api
uv sync              # or: python -m venv .venv
                     #     pip install -e ".[dev]"
```

Run the app as a module

```
python -m app      # uses app/__main__.py
```

Internal shared library

```
pip install git+https://github.com/acme/
    acme-auth.git@v2.3.0
```

Admin commands

```
python manage.py migrate   # Django CLI
typer / click for your own tools
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What does if __name__ == ”__main__” do?”

- Every module has **__name__**. When you **run** a file it’s **”__main__”**; when it’s **imported** it’s the module name.
- So the code under the guard runs only when the file is executed directly.
- Use it for demos, CLIs and scripts, so importing the module has no side effects.

> **Follow-up they ask next:** ”How does Python find a module when you import it?” (sys.modules cache first, then the folders in sys.path: the script’s folder, PYTHONPATH, site-packages, the stdlib).

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you manage dependencies in a Python project?”

- One **virtual environment** per project; never the system Python.
- Declare direct dependencies with ranges in **pyproject.toml**.
- **Lock** exact versions (uv.lock, poetry.lock or pip-tools) so CI and production match.
- Separate **dev** dependencies (pytest, mypy, ruff).
- Update regularly and scan for vulnerabilities (pip-audit, Dependabot).

Follow-up they ask next: ”What is the difference between a module and a package?” (a module is one .py file; a package is a folder of modules, usually with __init__.py).

RECAP REMEMBER THESE

Module 14 in 6 lines

1. Module = a .py file; package = a folder with **__init__.py**.
2. Imports are cached in **sys.modules** and searched on **sys.path**.
3. Guard script code with **if __name__ == "__main__"**.
4. Avoid circular imports: dependencies point one way.
5. **argparse** subcommands for CLIs; **[project.scripts]** makes a command.
6. One **venv** per project, **pyproject.toml**, a lock file.

PRACTICE NOW

Do exercises/ex14_wordtool_cli.py, then install tasky with pip install -e .